

AGENCYOS

Complete Development Roadmap & Definition of Done

Document 20 of the AgencyOS Master Documentation Set

CURRENT REPOSITORY • GAP ANALYSIS • PHASES • DEPENDENCIES • IMPLEMENTATION •
TESTING • DEPLOYMENT • PRODUCTION • FINAL COMPLETION

Purpose: Provide the implementation roadmap Claude/development agents can use to inspect the existing AgencyOS repository, identify what is missing, build the system in dependency order, verify every phase, deploy safely and determine—using objective evidence—when AgencyOS is genuinely complete.

1. Roadmap Core Principle

- Do not start by rewriting the repository.
- First inspect the current repository, architecture, database, routes, UI, integrations, tests, environment and deployment setup.
- Create an evidence-based gap analysis against Documents 01–19.
- Preserve working functionality unless there is a documented reason to change it.
- Build in dependency order.
- Every phase ends with verification evidence.
- A phase is not complete because code exists; it is complete only when its Definition of Done passes.
- Production completion requires the final system-wide Definition of Done.

2. Current Repository Discovery

- Repository structure and technology stack.
- Frontend application(s).
- Backend/API services.
- Database/schema/migrations.
- Authentication/authorization.
- Current Admin panel.
- Current client portal.
- CRM/sales functionality.
- Project management.
- Finance/invoicing/payment code.
- WhatsApp/communication integrations.
- AI provider integrations.
- Agent/orchestration code.
- UI/design/prototype workflow.
- Developer/CI/CD workflow.
- Testing infrastructure.
- Deployment/infrastructure.
- Environment variables and configuration.
- Existing documentation.
- Known TODOs, placeholders, mocks and unfinished features.

3. Repository Audit Rule

- Do not assume a feature is missing merely because it is not obvious from the UI.
- Trace feature implementation across UI, API, database, permissions, jobs, integrations and tests.
- Distinguish COMPLETE, PARTIAL, MOCKED, BROKEN, MISSING and UNKNOWN.
- Record evidence: file/module, route/API, schema, test, configuration or runtime behavior.
- Do not silently replace existing architecture with a preferred stack.

4. Gap Analysis Matrix

- Each requirement from Documents 01–19 receives a status.

- Status values: COMPLETE, PARTIAL, MISSING, BROKEN, MOCKED, NEEDS_REVIEW.
- Record current implementation.
- Record missing pieces.
- Record dependencies.
- Record risk.
- Record proposed implementation.
- Record validation method.
- Record target phase.
- Record owner/agent.

5. Phase Planning Rules

- Prioritize foundations before features.
- Do not implement downstream workflows against unstable data models.
- Security, tenancy and permissions are foundational.
- Shared entities and state machines must be defined before agent automation.
- Payments and deployment require stronger controls than ordinary CRUD.
- Client-facing visibility depends on authoritative backend records.
- Testing is built alongside features, not postponed to the end.

6. Phase 0 — Repository Audit & Baseline

- Clone/open current repository.
- Install dependencies using documented lockfiles.
- Run existing application.
- Run existing tests.
- Run lint/type/static checks where available.
- Inspect database and migrations.
- Inspect environment/configuration without exposing secrets.
- Map routes/APIs.
- Map major components/modules.
- Identify current deployment path.
- Create baseline build/test report.
- Create gap-analysis matrix.
- Do not make major architectural changes in Phase 0.

7. Phase 0 Definition of Done

- Repository builds or exact failure is documented.
- Application starts or exact blocker is documented.
- Existing tests are executed.
- Database state is understood.
- Environment requirements are documented.
- Architecture map exists.
- Gap matrix exists.
- Critical unknowns are identified.

- No secrets are copied into documentation or logs.

8. Phase 1 — Foundation & Architecture

- Establish canonical organization, user, project, client and membership models.
- Define service/module boundaries.
- Define API conventions.
- Define event/audit conventions.
- Define configuration management.
- Define error handling.
- Define background job strategy.
- Define file/storage abstraction.
- Define notification abstraction.
- Define state-machine conventions.
- Define shared IDs/versioning.

9. Phase 1 Definition of Done

- Core schema is migrated safely.
- Core APIs are consistent.
- Application boots from clean setup.
- Architecture decisions are documented.
- Core entities have tests.
- Errors are observable.
- No cross-organization data leakage is possible in tested paths.

10. Phase 2 — Authentication, Security & Tenant Isolation

- Implement authentication.
- Implement organization membership.
- Implement RBAC.
- Implement RLS/resource authorization where applicable.
- Implement client/project isolation.
- Implement session management.
- Implement secure file authorization.
- Implement audit logging.
- Implement secret-management integration.
- Implement permission middleware/policies.
- Add authorization tests.

11. Phase 2 Definition of Done

- All protected endpoints require authorization.
- Role permissions are tested.
- Cross-tenant access tests fail closed.
- Client cannot access internal records.
- Privileged actions are audited.

- Secrets are not exposed.
- Session revocation works.
- Security baseline passes.

12. Phase 3 — Admin Control Center & Policy Engine

- Admin dashboard.
- User/role management.
- Organization settings.
- Approval workflows.
- Pricing/discount policies.
- Payment exception policies.
- Deployment gates.
- AI model/provider configuration.
- API key management.
- Agent permissions.
- Feature flags.
- Audit/override controls.

13. Phase 3 Definition of Done

- Every supported policy has a UI/API/configuration path.
- Policy changes are audited.
- Overrides require permission.
- High-risk actions cannot bypass policy through ordinary UI/API calls.
- Configuration changes are versioned where required.

14. Phase 4 — AI Orchestrator & Multi-Model Routing

- Create provider abstraction.
- Support approved models/providers such as OpenAI/ChatGPT, Claude, Gemini, Grok, DeepSeek and future providers.
- Create agent registry.
- Define agent capabilities.
- Define model-routing policies.
- Define fallback rules.
- Define cost/quality/availability signals.
- Define context limits.
- Define provider health.
- Define task-level routing.
- Ensure agents never receive raw provider secrets unnecessarily.

15. Phase 4 Definition of Done

- Multiple providers can be configured.
- Routing is policy-driven.
- Fallback is controlled.
- Provider failures are observable.

- Agent permissions are enforced independent of model choice.
- Usage/cost metadata is tracked.
- Sensitive context policies are respected.

16. Phase 5 — Memory, Context & Agent Handoffs

- Project memory.
- Client history.
- Organization context.
- Agent task context.
- Conversation history.
- Shared artifacts.
- Version references.
- Agent handoff protocol.
- Memory access control.
- Context minimization.
- Memory retention/deletion.

17. Phase 5 Definition of Done

- Agents can resume work from structured context.
- Unauthorized project/client memory cannot be retrieved.
- Handoffs preserve required state.
- Memory records have ownership/scope.
- Sensitive memory is protected.
- Agent context is traceable.

18. Phase 6 — CRM, Sales & Lead Management

- Facebook/lead intake.
- Lead creation.
- Lead deduplication.
- WhatsApp/contact linkage.
- Qualification.
- Requirements discovery.
- Quote generation.
- Objection handling.
- Negotiation.
- Offer/discount policy.
- Conversion/lost-lead states.
- Sales-agent workflows.

19. Phase 6 Definition of Done

- Lead can move from acquisition to conversion/lost state.
- Duplicate handling works.
- Sales activity is auditable.

- Quote versions are linked.
- Approved discounts follow policy.
- Client communication is tracked.
- Converted lead creates a project/onboarding path.

20. Phase 7 — Client Onboarding & Project Management

- Client profile.
- Project creation.
- Project ownership.
- WhatsApp group workflow.
- Project members.
- Scope.
- Dates.
- Milestones.
- Tasks.
- Dependencies.
- Project state machine.
- Client onboarding checklist.

21. Phase 7 Definition of Done

- Converted lead can become a project.
- Project has authoritative scope/owner/status.
- Milestones/tasks are linked.
- Client access is isolated.
- Project lifecycle is auditable.

22. Phase 8 — Requirements, Scope & Change Requests

- Requirement collection.
- Requirement normalization.
- Feature extraction.
- Scope baseline.
- Scope freeze.
- Included revisions.
- Small/free changes.
- Large/paid changes.
- Change Request workflow.
- Impact analysis.
- Approval/versioning.

23. Phase 8 Definition of Done

- Approved scope has a unique version.
- New requests do not silently alter scope.
- Change requests can affect price/timeline.

- Approvals reference exact scope version.
- PM/Client/Sales/Finance receive consistent scope state.

24. Phase 9 — UI Design & Prototype

- AI UI Designer.
- Design system/theme.
- Screen inventory.
- Scope coverage.
- Design versions.
- Client review.
- Admin/PM approval.
- Prototype generation.
- Prototype review.
- Revision workflow.
- Approved UI baseline.

25. Phase 9 Definition of Done

- Every approved screen is versioned.
- Client approval references exact version.
- Prototype is linked to UI baseline.
- Revision history exists.
- Design-to-development handoff is structured.

26. Phase 10 — Development & AI Coding Agents

- Task decomposition.
- Dependency graph.
- Developer-agent assignment.
- Repository/worktree/branch strategy.
- Code generation/modification.
- Builds.
- Unit/integration/E2E tests.
- Code review.
- Self-verification.
- Retries/recovery.
- Artifact/version management.
- QA handoff.

27. Phase 10 Definition of Done

- Developer agents cannot mark completion without required evidence.
- Build passes.
- Required tests pass.
- Acceptance criteria map to implementation/tests.
- Code review passes.

- Exact commit/build is recorded.
- QA receives a reproducible artifact.

28. Phase 11 — QA, Security, Performance & Production Readiness

- 360° test plan.
- Functional testing.
- UI testing.
- API/database testing.
- Integration/E2E.
- Regression.
- Security.
- Performance.
- Compatibility.
- Bug severity.
- Readiness score.
- Hard deployment gates.
- Smoke tests.

29. Phase 11 Definition of Done

- Mandatory test categories pass.
- No unresolved release-blocking defects.
- Security gates pass.
- Performance gates pass.
- Readiness score meets configured threshold.
- Exact release candidate is identified.
- Rollback plan exists where required.

30. Phase 12 — Finance, Invoice, Payment & Milestones

- Quotes.
- Invoice generation.
- GST/non-GST configuration.
- Payment accounts.
- UPI/QR/bank transfer.
- Payment submission.
- Verification.
- Receipts.
- Reconciliation.
- Milestone financial state.
- Refund/exception workflows.

31. Phase 12 Definition of Done

- Financial records are versioned/audited.

- Payment cannot be marked verified without evidence.
- Milestone payment state is linked to project gates.
- Partial/over/under/unmatched payments are supported.
- Final balance/exception is known.

32. Phase 13 — Client Portal & Transparency

- Client dashboard.
- Progress.
- Requirements/scope.
- Design/prototype review.
- Approvals.
- Milestones.
- Invoices/payments.
- Deliverables.
- Handover.
- Notifications.
- Client roles.

33. Phase 13 Definition of Done

- Client sees accurate current state.
- Versions and approvals are explicit.
- Internal/private information is hidden.
- Client actions create structured workflow events.
- Portal data comes from authoritative records.

34. Phase 14 — Handover, Completion & Customer Success

- Final payment gate.
- Final QA.
- Production deployment.
- Smoke testing.
- Handover package.
- Secure access transfer.
- Client acceptance.
- Completion record.
- Customer Success handoff.
- Warranty/support.

35. Phase 14 Definition of Done

- Final payment verified or approved exception.
- Production validated.
- Handover complete.
- Client acceptance captured where required.
- Completion record references exact release.

- Customer Success receives structured account context.

36. Phase 15 — Maintenance, Retention & Upsell

- Maintenance plans.
- Support/warranty classification.
- Maintenance tickets.
- Renewals.
- Customer health.
- VIP criteria.
- Churn risk.
- Retention recovery.
- Upsell detection.
- Repeat projects.
- Sales handoff.

37. Phase 15 Definition of Done

- Completed clients can enter maintenance.
- Renewals are tracked.
- Health signals are explainable.
- New work is separated from maintenance.
- Upsell opportunities are evidence-based.
- Commercial approvals remain enforced.

38. Phase 16 — Security, Audit & Data Governance Hardening

- Full security review.
- RLS/tenant isolation audit.
- Permission matrix audit.
- Secret rotation.
- API security review.
- AI permission review.
- Audit-log coverage.
- Retention policies.
- Deletion controls.
- Backup verification.
- Restore testing.
- Incident response.

39. Phase 16 Definition of Done

- Security tests pass.
- No known critical tenant-isolation failure.
- Secrets are protected.
- Audit coverage meets policy.
- Backups are successful.

- Restore test succeeds.
- Retention/deletion policies are operational.

40. Phase 17 — Integrated System Testing

- Test the complete lifecycle end to end.

Facebook lead → WhatsApp → Sales → Quote → Onboarding → Scope → UI → Prototype → Development → QA → Invoice → Payment → Production → Handover → Customer Success → Maintenance/Upsell.

- Test happy paths.
- Test partial payments.
- Test client rejection/revision.
- Test scope changes.
- Test agent failures.
- Test provider failures/fallback.
- Test deployment failure/rollback.
- Test permission failures.
- Test duplicate events.
- Test data recovery.

41. Phase 17 Definition of Done

- Critical end-to-end workflows pass.
- Cross-module state remains consistent.
- Failures recover or escalate correctly.
- No workflow relies on an unverified manual assumption.
- All critical transitions are auditable.

42. Phase 18 — Staging / UAT

- Deploy production-like staging.
- Run final regression.
- Run client acceptance scenarios.
- Run Admin/Finance/PM workflows.
- Validate notifications.
- Validate payment test flows.
- Validate AI provider routing.
- Validate backups/recovery.
- Capture UAT defects.
- Freeze release candidate after approval.

43. Phase 18 Definition of Done

- UAT acceptance achieved.
- All blocking defects resolved.
- Release candidate frozen.
- Deployment checklist approved.

- Rollback tested/validated where required.

44. Phase 19 — Production Deployment

- Production infrastructure ready.
- Secrets configured securely.
- Database migrations executed safely.
- Release candidate deployed.
- Smoke tests run.
- Monitoring enabled.
- Alerts enabled.
- Critical workflow verified.
- Deployment recorded.
- Client-facing availability confirmed.

45. Phase 19 Definition of Done

- Production is running the approved candidate.
- Smoke tests pass.
- Critical metrics are healthy.
- No critical runtime errors.
- Monitoring/alerts are active.
- Rollback remains available according to policy.

46. Phase 20 — Post-Production Stabilization

- Monitor initial production period.
- Track errors.
- Track performance.
- Track support issues.
- Validate external integrations.
- Fix release defects through controlled patch workflow.
- Run regression after fixes.
- Document incidents.
- Transition to normal operations.

47. Phase 20 Definition of Done

- Stabilization period passes configured criteria.
- No unresolved critical production incident.
- Monitoring is healthy.
- Support ownership is assigned.
- Known limitations are documented.

48. Final System Definition of Done

- All required roadmap phases are complete.
- All mandatory requirements from Documents 01–19 are implemented or explicitly approved as out of scope.

- Repository contains no critical placeholder/mock path in production workflows.
- All critical workflows pass end-to-end.
- Security/tenant isolation passes.
- Finance/payment controls pass.
- AI routing and agent permissions pass.
- Client portal permissions pass.
- QA/production gates pass.
- Production deployment succeeds.
- Post-deployment smoke tests pass.
- Backups and restore verification pass.
- Audit logs cover required sensitive actions.
- Documentation is complete.
- Operations/support ownership exists.
- Final Admin approval is recorded.
- Final production version/build is tagged and reproducible.

49. No-False-Completion Rule

- Code written ≠ feature complete.
- Feature working locally ≠ production ready.
- Build passing ≠ project complete.
- Tests passing ≠ client acceptance.
- Payment received ≠ project complete.
- Client saying 'okay' ≠ formal approval when formal approval is required.
- Deployment succeeded ≠ handover complete.
- High readiness score ≠ release approval if a hard gate fails.
- All agents saying complete ≠ system complete.
- Completion requires the documented Definition of Done and evidence.

50. Final Evidence Package

- Repository commit/tag.
- Build artifact.
- Environment/version record.
- Architecture document.
- Database migration state.
- Test reports.
- Security report.
- Performance report.
- Defect report.
- Readiness score.
- Deployment log.
- Smoke-test result.
- Backup/restore evidence.
- Client acceptance.

- Financial close.
- Handover checklist.
- Customer Success handoff.

51. Roadmap Execution Rules for Claude

- Start every implementation session by inspecting the current repository and current phase status.
- Do not assume prior work exists; verify it.
- Before changing architecture, explain dependency and migration impact internally through the project workflow.
- Prefer incremental changes over destructive rewrites.
- Create tests alongside features.
- Run targeted tests after each meaningful change.
- Run broader regression before phase completion.
- Never mark a phase complete without its Definition of Done evidence.
- Keep a machine-readable implementation status.
- Keep unresolved blockers visible.
- Do not hide failures by weakening tests.
- Do not remove a failing test merely to make CI green.
- Do not use mock success paths in production.
- Do not commit secrets.

52. Phase Tracking Record

- Phase ID.
- Phase name.
- Status.
- Dependencies.
- Started.
- Completed.
- Owner/agent.
- Repository commit.
- Tests.
- Known issues.
- Definition-of-Done status.
- Approval.
- Notes.

53. Recommended Phase Statuses

- NOT_STARTED.
- DISCOVERY.
- IN_PROGRESS.
- BLOCKED.
- READY_FOR_REVIEW.
- QA.
- READY_FOR_DEPLOYMENT.

- DEPLOYING.
- STABILIZING.
- COMPLETE.
- DEFERRED_BY_APPROVED_DECISION.

54. Dependency Rules

- Security/identity precedes sensitive workflows.
- Core data models precede dependent features.
- Scope precedes design approval.
- Design/prototype baseline precedes UI-to-code acceptance where applicable.
- Development precedes final QA.
- QA precedes production.
- Finance/payment verification controls milestone gates.
- Production validation precedes completion.
- Completion precedes normal post-project retention workflows.

55. Final Roadmap Acceptance Criteria

- Current repository is inspected before implementation.
- Missing features are identified from evidence.
- Phases are dependency-ordered.
- Every phase has a Definition of Done.
- Tests are integrated into development.
- Security is foundational and continuously tested.
- Financial and deployment gates are enforced.
- AI agents cannot bypass permissions or completion gates.
- Production deployment is reproducible.
- Post-production validation exists.
- Final completion is evidence-based.
- System can move from lead to completed project to maintenance/upsell without breaking state or authorization.

56. Master Roadmap

Phase	Primary outcome	Depends on
0	Repository audit + gap baseline	Current repository
1	Foundation/data architecture	0
2	Security/auth/RLS	1
3	Admin + policy engine	1–2
4	AI orchestrator + multi-model routing	2–3
5	Memory/context/handoffs	4
6	CRM/Sales/Leads	1–5
7	Onboarding/PM	1–3,6
8	Requirements/scope/change requests	7
9	UI/design/prototype	8

10	Development/coding agents	8–9
11	QA/security/performance readiness	10,2
12	Finance/payment/milestones	7–8,2
13	Client portal	7–12
14	Handover/completion/customer success	11–13
15	Maintenance/retention/upsell	14
16	Security/governance hardening	2–15
17	Integrated E2E testing	6–16
18	Staging/UAT	17
19	Production deployment	18
20	Stabilization/operations	19

57. Final Architecture

CURRENT REPOSITORY → AUDIT/GAP ANALYSIS → FOUNDATION → SECURITY → ADMIN/POLICY → AI ORCHESTRATION → CRM → PROJECT/REQUIREMENTS → DESIGN/PROTOTYPE → DEVELOPMENT → QA/READINESS → FINANCE → CLIENT PORTAL → HANDOVER → CUSTOMER SUCCESS → MAINTENANCE/UPSELL → HARDENING → E2E → UAT → PRODUCTION → STABILIZATION → FINAL COMPLETION

Key outcome: Claude should not simply 'build features'. It should operate against this roadmap, inspect what already exists, implement only the missing/defective pieces, verify each phase, preserve evidence, and refuse to declare AgencyOS complete until the final Definition of Done is objectively satisfied.